SCHOOL OF COMPUTER SCIENCE · UPES DEHRADUN

SpeakQL

Build Plan · Weeks 1–7, serialised

The Backend Plan's build order is written as three parallel lanes. There is one person. This is the same seven weeks, serialised, with the dependencies that decide what has to come first and the tests that decide when each phase is finished. It plans the mid-term presentation; §6 states what the end-term inherits.

Working document. Sequenced from Backend Plan §23 and the synopsis PERT chart.

Minor Project · MCA (AI & ML)
Garv Bhargava · Dhruv Chhatrawal · Kritant Poudnel
Mentor: Mr. Pankaj Dadure

1. The decision

PRESENTATION	SCOPE	WHAT THAT MEANS
Mid-term	Build weeks **1–7**, complete, all three lanes	Both models trained, the full twelve-step pipeline running, and the corrections system including the member overlay and the merge queue. Roughly **80%** of the project.
End-term	Build weeks **8–10**	Streaming, voice, exports, the evaluation harness, the ablation table and the threshold sweep. A **results** presentation, which is the right shape for a final.

An earlier plan deferred both CodeT5 and the LLM to the end-term. That was revised: the mid-term now carries the whole build through week 7, and the end-term becomes the measurement. The consequence is that **synthetic pair generation and Model B move inside the mid-term window**, which is what §3 is about.

2. The shape of the problem

Five things happen in a fixed order, and each one waits on the one before it. This is the critical path, and it is quoted from the synopsis PERT slide rather than invented here.

```
data prep -> SYNTHETIC PAIRS -> Model B -> router -> evaluation
  W1-2 the gate CodeT5 needs two W8-9
                                  two stages generators
```

> **The frontend is not on this chain, and the deck says so.**
> "Critical path: data preparation → synthetic pair generation → Model B training → router → evaluation. Front-end work runs in parallel and is not on the critical path." The finished interface, the dark theme and the three specifications are all on the parallel track. They are genuinely ahead of schedule, and none of them shorten this chain by a single day.

The arithmetic worth knowing. Seven weeks across three lanes is **21 lane-weeks** of work. The finished frontend removes roughly three to four of those from Lane B, leaving about **17–18 lane-weeks for one person**. That is the number the schedule has to absorb, and it is why the order in §3 matters more than it would for a team of three.

3. The build order — eight phases

Numbered because the order is real: each phase unlocks the next. A phase is finished when its stated test passes, not when the code is written.

3.1 Foundation

Lane B + C · weeks 1–2

- Repo skeleton, `docker compose`, health endpoint.
- Warehouse DDL and **seed both organisations** — Northwind and a second tenant, because isolation cannot be tested against one.
- The five Postgres roles, and the **six privilege assertions** running in CI.

> **Done when.**
> One `docker compose up` gives a healthy API and a seeded warehouse, and the six privilege assertions pass.

3.2 Open the gate — synthetic pairs

Start on day one · runs for weeks

- Write the pair generator against the project's own 18-table schema.
- **Execution-verify every pair** — run it, and keep it only if it returns.
- Target about **1,500 verified in-domain pairs**.

> **Done when.**
> The verified pair count climbs daily and the verifier rejects bad pairs without supervision. This phase is not "finished" before Model B needs it; it is *running*.

3.3 Model A — retrieval

About 25 minutes to train

- Spider preparation.
- MiniLM bi-encoder fine-tune, with **in-schema hard negatives**.
- Schema introspection into `schema_registry`, then the FAISS index.

> **Done when.**
> **recall@10 is a number that can go on a slide.** It is the first measured result in the project and it costs less than half an hour of training.

3.4 The spine — auth and permission

Lane B · weeks 2–4

- Two-purpose OTP, lockout, JWT, domain resolution, **personal workspaces**.
- The signup question, invitations, grants, `object_access.py`.
- Organisation and people routes — and **freeze the §12 API contract here**.

> **Done when.**
> The finished frontend's existing calls hit real endpoints, and the isolation and object-access tests are green. The interface is **already built against this contract**, which is what makes the frontend free once it is frozen.

3.5 Model B — the generator

Three to four hours per stage

- Stage one on Spider.
- Stage two on Spider **plus the verified pairs from §3.2**.

> **Done when.**
>
> Execution accuracy is measured on the held-out benchmark. **The training is the short part** — if §3.2 started on day one, this is two afternoons.

3.6 Safety and execution

Lane A · weeks 6–7

- The sqlglot **AST validator**, and the edit-validator for the write path.
- Router and confidence normalisation — length-normalised beam score, threshold 0.55.
- Executor, including overlay resolution, under the read-only role.
- `/api/ask` end to end, **proved by curl before any interface is attached**.

> **Done when.**
>
> **All five refusal modes reproduce from curl**, and the adversarial set — including the prompt-injection cells — passes without changing the explainer's behaviour.

3.7 Corrections

The hard one

- Owner path: writes the real table immediately, one `edit_log` row, no approval step.
- **Member overlay and its `COALESCE` read-back views** — the executor resolves a member's own reads through their overlay, and nobody else's.
- Merge requests, and **the staleness check**: re-read the live row, compare it with the before-value the request was raised against, and refuse if it moved.

> **Done when.**
>
> The **real-table checksum test** proves the warehouse is byte-identical after a member's edit. Revision 5 calls the staleness case a real data-loss bug rather than an omission, so budget more time here than the line count suggests.

3.8 Ingestion and connections

Lane B · week 5

- File ingestion that **asks where the data should live**, then reindexes.
- External connection registration with the host guard — resolve the name, refuse a private range, require TLS, and verify the supplied account cannot write.

> **Done when.**
>
> An uploaded CSV is **queryable immediately**, with no manual reindex step. The demonstration script depends on this.

3.9 Why §3.2 is second and not third

Model B waits on pairs. The router waits on Model B. Evaluation waits on the router. **Everything after that point is serial**, which makes pair generation the one task that should be running in the background from day one while the product surface is built. It is the single change to this plan that most reduces schedule risk.

Tests interleave. They are never saved for the end.

Isolation and object-access tests land with the grants in §3.4. RBAC and host-guard tests with §3.8. Overlay, staleness and checksum tests with §3.7. The adversarial set with §3.6. The Frontend Specification already carries **132 numbered cases, 106 of them automated** — they are a checklist to build against, not a final exam.

4. If the schedule slips

The cut order is fixed by the synopsis deck and is not re-decided here.

ORDER	WHAT GOES	WHAT SURVIVES IT
Cut 1st	Chart-type classifier	A rule on the result shape picks the chart instead
Cut 2nd	Question gatekeeper	The AST validator still refuses everything dangerous — this is a layer of convenience, not the layer of safety
Cut 3rd	Admin approval console	Approvals happen, without a dedicated screen
Never	The validator · tenant isolation · **the ablation table**	Named in the deck as never cut

The ablation table cannot exist without Model B.

It compares CodeT5 alone, Gemma alone, and the hybrid routed at 0.55. The deck lists it among the things that are never cut, which makes §3.2 and §3.5 the **last** items on this plan to abandon rather than the first. Cutting the pair pipeline to save time quietly cuts the results chapter with it.

5. What the end-term inherits

WEEK	LANE A — MODELS AND PIPELINE	LANE B — PRODUCT AND INTERFACE	LANE C — DATA, EVALUATION, TESTS
8	Streaming endpoint and the §12.6 envelope; fallback wiring	Voice, export in both forms, notifications, feedback	**Evaluation harness; ablation runs begin**
9	**Threshold sweep; ablation completed**	Frontend integration across both roles	Failure analysis; full test suite green
10	Model cards, reproduction notes	`make demo` green end to end	Results tables; report amendments

This is a results presentation, and that is the right shape for a final. The mid-term shows a system that runs; the end-term shows what measuring it revealed — the threshold picked off a curve rather than asserted, and five configurations compared.

6. What exists today

STATE	WHAT
Built	The complete interface: every screen, both roles, all five refusal modes, both correction paths, dark by default, published and walkable in 34 steps. Three specifications — Backend Plan (31pp), Complete Logic (18pp), Frontend Specification (42pp) — carrying 132 numbered test cases.
Not yet written	Application code. Every phase in §3 starts from an empty repository, which is why §3.1 is a day of plumbing before anything interesting happens, and why freezing the API contract in §3.4 is worth the hour it takes.

> **One number is still missing from this plan: the date.**
>
> Everything above is an **order**, and the order holds whatever the calendar says. Fill in the weeks remaining and it becomes a **schedule** — and the first thing to check is whether §3.2 has enough runway ahead of §3.5.